

Agenda

- Pakiety
- Deployment
- Maven

Pakiety



Importowanie klas z pakietów

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();

        list.addAll(List.of(1, 2, 3, 4, 5));
        list.forEach(System.out::println);
    }
}
```

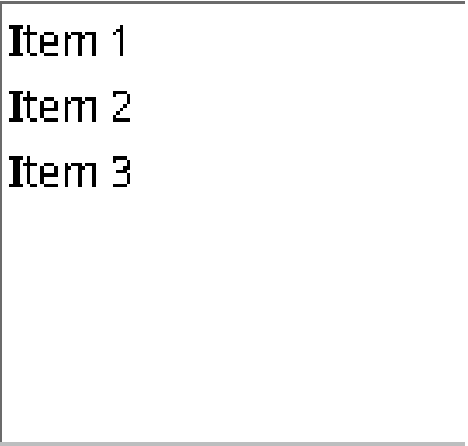
Importowanie całych pakietów

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
  
        List<Integer> list = new ArrayList<>();  
  
        list.addAll(List.of(1, 2, 3, 4, 5));  
        list.forEach(System.out::println);  
    }  
}
```

Widget listy

```
import java.awt.Frame;  
import java.awt.List;  
  
public class Main {  
    public static void main(String[] args) {  
        Frame frame = new Frame("List");  
  
        List list = new List();  
  
        list.add("Item 1");  
        list.add("Item 2");  
        list.add("Item 3");  
  
        frame.add(list);  
  
        frame.setVisible(true);  
    }  
}
```



Item 1
Item 2
Item 3

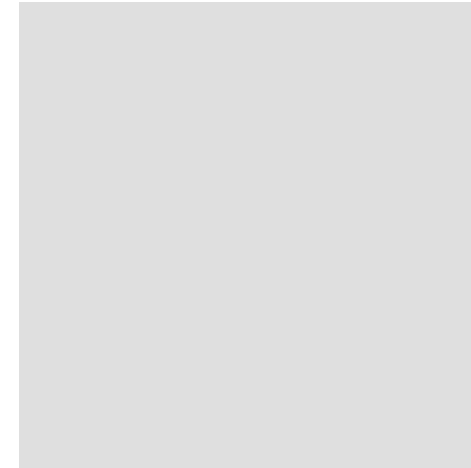
Konflikt nazw

```
import java.awt.*; // java.awt.List
import java.util.*; // java.util.List

public class Main {
    public static void main(String[] args) {

        Frame frame = new Frame("List");

        frame.setVisible(true);
    }
}
```



Konflikt nazw

```
import java.awt.*; // java.awt.List
import java.util.*; // java.util.List

public class Main {
    public static void main(String[] args) {
        List<String> items = List.of("Item 1", "Item 2", "Item 3");

        Frame frame = new Frame("List");
        List list = new List();
        items.forEach(list::add);

        frame.add(list);
        frame.setVisible(true);
    }
}
```

Ujednoznacznienie nazwy klasy

```
import java.awt.*; // java.awt.List
import java.util.*; // java.util.List

public class Main {
    public static void main(String[] args) {
        java.util.List<String> items = java.util.List.of("Item 1", "Item 2", "Item 3");

        Frame frame = new Frame("List");
        java.awt.List list = new java.awt.List();
        items.forEach(list::add);

        frame.add(list);
        frame.setVisible(true);
    }
}
```


Bezwzględna ścieżka do klasy

```
public class Main {  
    public static void main(String[] args) {  
        java.util.List<String> items = java.util.List.of("Item 1", "Item 2", "Item 3");  
  
        Frame frame = new java.awt.Frame("List");  
        java.awt.List list = new java.awt.List();  
        items.forEach(list::add);  
  
        frame.add(list);  
        frame.setVisible(true);  
    }  
}
```

Wyróżnienie importu klasy

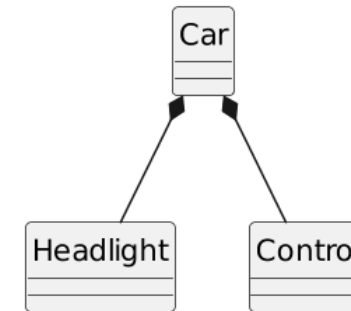
```
import java.awt.*;
import java.awt.List;
import java.util.*;

public class Main {
    public static void main(String[] args) {
        java.util.List<String> items = java.util.List.of("Item 1", "Item 2", "Item 3");
        Frame frame = new Frame("List");
        List list = new List();
        items.forEach(list::add);
        frame.add(list);
        frame.setVisible(true);
    }
}
```

Definicja przynależności do pakietu

```
package car;
```

```
public class Car {  
    private Light[] headlight = new Light[2];  
    private Control lightControl = new Control();  
    private Control errorControl = new Control();  
}
```



Struktura projektu

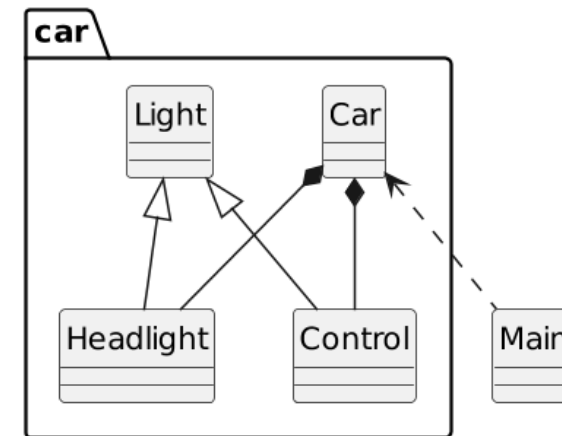
```
package car;  
public class Car { /*...*/ }
```

```
package car;  
public class Light { /* ... */ }
```

```
package car;  
public class Headlight extends Light { }
```

```
package car;  
public class Control extends Light { }
```

```
import car.Car;  
  
public class Main {  
    public static void main(String[] args) {  
        Car car = new Car();  
    }  
}
```



Dostęp pakietowy wewnątrz pakietu

```
package car;

public class Light {
    boolean isOn = false;

    void setOn(boolean value) {
        isOn = value;
    }

    public void setOn() {
        isOn = true;
    }

    public void setOff() {
        isOn = false;
    }
}
```

```
package car;

public class Car {
    private Light[] headlight = new Light[2];
    private Control lightControl = new Control();
    private Control errorControl = new Control();

    public void headlightOn() {
        Arrays.stream(headlight).forEach(Light::setOn);
    }

    public void headlightOff() {
        Arrays.stream(headlight).forEach(Light::setOff);
    }

    private void getStatus() {
        if(headlight[0] != headlight[1])
            errorControl.setOn();
        else
            lightControl.setOn(headlight[0].isOn);
    }
}
```

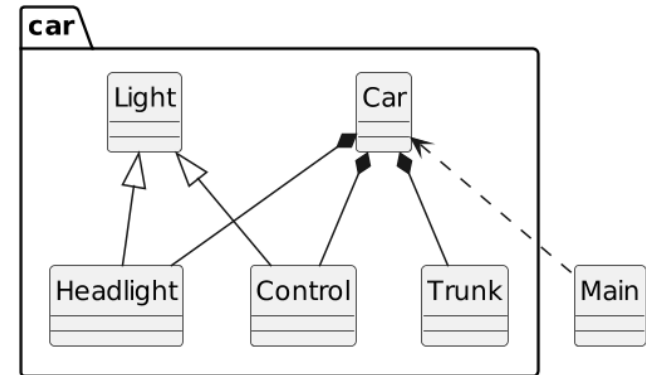
Klasa o dostępie pakietowym

```
package car;

class Trunk {
    private List<Object> items = new ArrayList<>();

    public List<Object> getItems() {
        return items;
    }

    public void addItem(Object item) {
        items.add(item);
    }
}
```



Klasa o dostępie pakietowym

```
package car;

public class Car {
    /* ... */
    private Trunk trunk;

    public Trunk getTrunk() {
        return trunk;
    }

    public List<Object> thingsFromTheTrunk() {
        return trunk.getItems();
    }

    public void putInTheTrunk(Object object) {
        trunk.addItem(object);
    }
}
```

```
import car.Car;

public class Main {
    public static void main(String[] args) {
        Car car = new Car();

        car.putInTheTrunk("things");
        car.thingsFromTheTrunk()
            .forEach(System.out::println);
    }
}
```

Widoczność

	default	public	protected	private
Ta sama klasa	✓	✓	✓	✓
Klasa w tym samym pakiecie	✓	✓	✓	X
Klasa dziedzicząca w tym samym pakiecie	✓	✓	✓	X
Klasa dziedzicząca w innym pakiecie	X	✓	✓	X
Klasa w innym pakiecie	X	✓	X	X

Nazewnictwo

```
package pl.umcs.oop.car;  
  
public class Car {  
    /* ... */  
    private Trunk trunk;  
  
    public Trunk getTrunk() {  
    return trunk;  
    }  
  
    public List<Object> thingsFromTheTrunk() {  
        return trunk.getItems();  
    }  
  
    public void putInTheTrunk(Object object) {  
        trunk.addItem(object);  
    }  
}
```

Deployment



Kompilacja

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

```
$ ls  
Main.java
```

```
$ javac Main.java  
$ ls  
Main.class Main.java
```

```
$ java Main  
Hello world!
```

Kompilacja wieloplikowa

```
public class Main {  
    public static void main(String[] args) {  
        Another another = new Another();  
        another.printText();  
    }  
}
```

```
public class Another {  
    void printText() {  
        System.out.println("Text");  
    }  
}
```

```
$ javac Another.java Main.java
```

```
$ java Main  
Text
```

```
$ java Another
```

```
Error: Main method not found in class Another, please define the main method as:  
    public static void main(String[] args)  
or a JavaFX application class must extend javafx.application.Application
```

Wiele metod main w ramach projektu

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

```
public class Another {  
    public static void main(String[] args) {  
        System.out.println("Another world!");  
    }  
}
```

```
$ javac Another.java Main.java
```

```
$ java Main  
Hello world!
```

```
$ java Another  
Another world!
```

Argumenty linii komend

```
public class Main {  
    public static void main(String[] args) {  
        Arrays.stream(args).forEach(System.out::println);  
    }  
}
```

```
$ javac Main.java
```

```
$ java Main
```

```
$ java Main one two three "four five"
```

```
one
```

```
two
```

```
three
```

```
four five
```

Tworzenie pliku JAR

```
public class Main {  
    public static void main(String[] args) {  
        Another another = new Another();  
        another.printText();  
    }  
}
```

```
public class Another {  
    void printText() {  
        System.out.println("Text");  
    }  
}
```

```
$ javac Another.java Main.java
```

```
$ jar cf archive.jar Another.java Main.java
```

```
$ ls
```

```
Another.class  Another.java  Main.class  Main.java  archive.jar
```

```
$ java -jar archive.jar
```

```
no main manifest attribute, in archive.jar
```

Wybór głównej klasy

```
public class Main {  
    public static void main(String[] args) {  
        Another another = new Another();  
        another.printText();  
    }  
}
```

```
public class Another {  
    void printText() {  
        System.out.println("Text");  
    }  
}
```

```
$ javac Another.java Main.java
```

```
$ jar cfe archive.jar Main Another.class Main.class
```

```
$ java -jar archive.jar  
Text
```


Zawartość pliku JAR

```
public class Main {  
    public static void main(String[] args) {  
        Another another = new Another();  
        another.printText();  
    }  
}
```

```
public class Another {  
    void printText() {  
        System.out.println("Text");  
    }  
}
```

```
$ jar tf archive.jar  
META-INF/  
META-INF/MANIFEST.MF  
Main.class  
Another.class
```

```
$ unzip -p archive.jar META-INF/MANIFEST.MF  
Manifest-Version: 1.0  
Created-By: 22 (N/A)  
Main-Class: Main
```

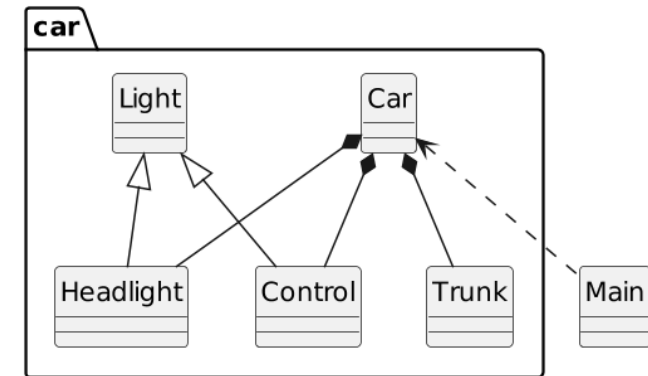
Określenie miejsca kompilacji

```
$ ls -R
.:
Main.java  car

./car:
Car.java  Control.java  Headlight.java
Light.java  Trunk.java

$ javac car/*.java -d ../out
$ ls -R ../out
.:
car

./car:
Car.class  Control.class  Headlight.class  Lig
ht.class  Trunk.class
```



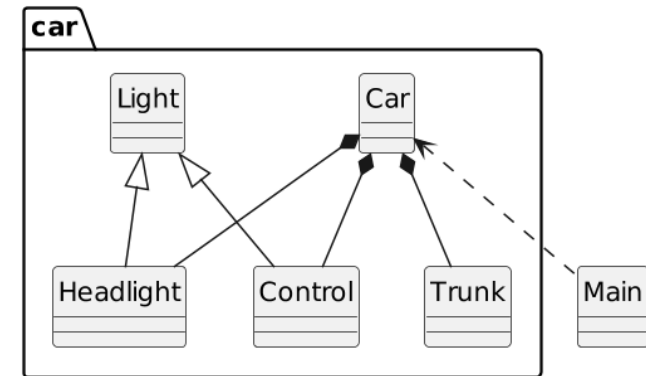
Plik JAR o charakterze biblioteki

```
$ ls -R ../out
.:
car

./car:
Car.class Control.class Headlight.class Lig
ht.class Trunk.class

$ cd ../out
$ jar cf car.jar car/*.class

$ ls
car car.jar
```



Domyślna ścieżka klas

```
$ cd ../src
$ javac Main.java -d ../out
$ cd ../out
$ ls -R ../out
.:
car  car.jar  Main.class

./car:
Car.class  Control.class  Headlight.class  Lig
ht.class  Trunk.class

$ java Main
things
```

```
import car.Car;

public class Main {
    public static void main(String[] args) {
        Car car = new Car();

        car.putInTheTrunk("things");
        car.thingsFromTheTrunk()
            .forEach(System.out::println);
    }
}
```

Classpath

```
$ mkdir /tmp/out
$ cp Main.class car.jar /tmp/out
$ cd /tmp/out

$ ls
car.jar  Main.class
#
$ java Main
Exception in thread "main" java.lang.NoClassDefFoundError: car/Car
    at Main.main(Main.java:6)
Caused by: java.lang.ClassNotFoundException: car.Car ...

$ java -cp .:car.jar Main
things
```

Obsługa bazy danych

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        try (
            Connection connection = DriverManager.getConnection("jdbc:sqlite:sample.db");
            Statement statement = connection.createStatement();
        ) {
            statement.executeUpdate("DROP TABLE IF EXISTS person");
            statement.executeUpdate("CREATE TABLE PERSON (id INTEGER, name STRING)");
            statement.executeUpdate("INSERT INTO PERSON VALUES(1, 'leo')");
            statement.executeUpdate("INSERT INTO PERSON VALUES(2, 'yui')");
            ResultSet rs = statement.executeQuery("SELECT * FROM person");
            while (rs.next()) {
                System.out.print("name = " + rs.getString("name"));
                System.out.print("id = " + rs.getInt("id"));
            }
        } catch (SQLException e) { e.printStackTrace(System.err); }
    }
}
```

źródło: <https://github.com/xerial/sqlite-jdbc>

Obsługa bazy danych

Usage

➔ More usage examples and configuration are available in [USAGE.md](#)

SQLite JDBC is a library for accessing SQLite databases through the JDBC API. For the general usage of JDBC, see [JDBC Tutorial](#) or [Oracle JDBC Documentation](#).

1. [Download](#) `sqlite-jdbc-3.45.3.0.jar` then append this jar file into your classpath.
2. [Download](#) `slf4j-api-1.7.36.jar` then append this jar file into your classpath.
3. Open a SQLite database connection from your code. (see the example below)

```
$ javac Main.java
$ java -cp slf4j-api-1.7.36.jar:sqlite-jdbc-3.45.3.0.jar: Main

name = leo id = 1 name = yui id = 2
```


Maven



Archetyp - podstawa nowego projektu

Maven Quickstart Archetype

`maven-archetype-quickstart` is an archetype which generates a sample Maven project:

```
1. project
2. |-- pom.xml
3. `-- src
4.     |-- main
5.     |   |-- java
6.     |       |-- $package
7.     |           |-- App.java
8.     `-- test
9.         |-- java
10.         |-- $package
11.             |-- AppTest.java
```

```
$ mvn \
  archetype:generate \
    -DarchetypeGroupId=\
      pl.umcs \
    -DartifactId=\
      sqlapp \
    -DarchetypeArtifactId=\
      maven-archetype-quickstart \
    -DinteractiveMode=\
      false
```

Project object model

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd>
  <modelVersion>4.0.0</modelVersion>

  <groupId>pl.umcs</groupId>
  <artifactId>sqlapp</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>sqlapp</name>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>22</maven.compiler.source>
    <maven.compiler.target>22</maven.compiler.target>
  </properties>

  <!-- ... -->
</project>
```

Etapy cyklu budowania

- validate
- compile
- test
- package
- verify
- install
- deploy

Kompilacja

```
$ mvn compile
```

```
$ tree target
```

```
target
```

```
├── classes
```

```
    ├── pl
```

```
        ├── umcs
```

```
            └── App.class
```

```
$ cd target/classes/pl/umcs
```

```
$ java App
```

```
Error: Could not find or load main class App
```

```
Caused by: java.lang.NoClassDefFoundError: App (wrong name: pl/umcs/App)
```

```
$ cd ../../..
```

```
$ java pl.umcs.App
```

```
Hello World!
```

Tworzenie pakietu

```
$ mvn package
$ tree target
target
└─ sqlapp-1.0-SNAPSHOT.jar

$ cd target
$ java -jar sqlapp-1.0-SNAPSHOT.jar
no main manifest attribute, in sqlapp-1.0-SNAPSHOT.jar

$ java -cp sqlapp-1.0-SNAPSHOT.jar pl.umcs.App
Hello World!
```

Wybór głównej klasy

```
<plugins>
  <plugin>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.2</version>
    <!-- ... -->
    <configuration>
      <archive>
        <manifest>
          <addClasspath>true</addClasspath>
          <mainClass>pl.umcs.App</mainClass>
        </manifest>
      </archive>
    </configuration>
  </plugin>
  <!-- ... -->
</plugins>
```

Aktualizacja pakietu

```
$ mvn package
$ tree target
target
└─ sqlapp-1.0-SNAPSHOT.jar

$ cd target
$ java -jar sqlapp-1.0-SNAPSHOT.jar
Hello World!
```

Obsługa bazy danych

```
import java.sql.*;

public class App {
    public static void main(String[] args) {
        try {
            Connection connection = DriverManager.getConnection("jdbc:sqlite:sample.db");
            Statement statement = connection.createStatement();
        } {
            statement.executeUpdate("DROP TABLE IF EXISTS person");
            statement.executeUpdate("CREATE TABLE PERSON (id INTEGER, name STRING)");
            statement.executeUpdate("INSERT INTO PERSON VALUES(1, 'leo')");
            statement.executeUpdate("INSERT INTO PERSON VALUES(2, 'yui')");
            ResultSet rs = statement.executeQuery("SELECT * FROM person");
            while (rs.next()) {
                System.out.print("name = " + rs.getString("name"));
                System.out.print("id = " + rs.getInt("id"));
            }
        } catch (SQLException e) { e.printStackTrace(System.err); }
    }
}
```


Repozytorium pakietów

MVN REPOSITORY

Search for groups, artifacts, categories

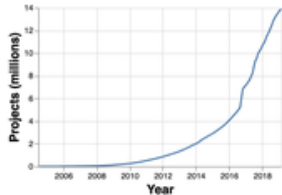
Search

Categories

Popular

Contact Us

Indexed Artifacts (35.3M)



Popular Categories

Testing Frameworks & Tools

Android Packages

Logging Frameworks

Java Specifications

JSON Libraries

JVM Languages

Language Runtime

Core Utilities

Mocking

Web Assets

Annotation Libraries

Home » org.xerial » sqlite-jdbc

xerial.org

SQLite JDBC

SQLite JDBC is a library for accessing and creating SQLite database files in Java (it includes native libraries)

License	Apache 2.0
Categories	JDBC Drivers
Tags	sqlite database sql jdbc driver rdbms
Ranking	#383 in MvnRepository (See Top Artifacts) #5 in JDBC Drivers
Used By	1,269 artifacts

Central (86)

Homer-Core (1)

	Version	Vulnerabilities	Repository	Usages	Date
3.45.x	3.45.2.0		Central	37	Mar 13, 2024
	3.45.1.0		Central	43	Jan 31, 2024
	3.45.0.0		Central	18	Jan 19, 2024
3.44.x	3.44.1.0		Central	71	Nov 27, 2023
	3.44.0.0		Central	20	Nov 13, 2023

Dodanie zależności i uruchomienie

```
<dependencies>
  <!-- ... -->
  <dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.45.2.0</version>
  </dependency>
</dependencies>
```

```
$ mvn package
$ java -jar sqlapp-1.0-SNAPSHOT.jar

name = leo id = 1 name = yui id = 2
```